

Benchmarking High-Throughput LLM Serving Frameworks

vLLM and SGLang improve concurrency-16 throughput by up to 39.1x over a HuggingFace baseline on shared-prefix prompts.

LLM Serving Benchmark

Yimeng Ye - yy3608

Can Yang - cy2811

Yuxuan Wang - yw4609

Jiawei Wang - jw4807

Zhijing Wu - zw3155

GitHub: https://github.com/lennonYe/6998_project

Motivation & Problem

Why LLM serving needs systems optimization.

Why this matters

- Interactive LLM products need low latency while serving many concurrent users.
- Naive generation underutilizes GPU capacity and repeats expensive prefill work.
- KV-cache memory management and request scheduling dominate serving efficiency.

Problem statement

Compare HuggingFace, vLLM, and SGLang for autoregressive inference under controlled concurrent workloads.

Goal: Measure TTFT, throughput, latency, and GPU memory across concurrency levels and prompt structures.

Technical Approach - Model & Data

Controlled serving benchmark across models, prompts, and concurrency levels.

Models

Qwen2.5-1.5B-Instruct, Qwen2.5-3B-Instruct, Llama-3.2-1B-Instruct, Qwen2.5-1.5B-AWQ-INT4.

Workload

Synthetic short Q&A prompts plus shared-prefix code-analysis prompts; 16 prompts per set to avoid cycling at max concurrency.

Hardware

NVIDIA L4 24 GB rerun; earlier mid-report numbers used NVIDIA L40S 48 GB.

Technical Approach - Optimizations Applied

We compare serving systems rather than changing model architecture.

1. vLLM / PagedAttention

Paged KV-cache management enables high-throughput continuous batching under concurrent requests.

2. SGLang / RadixAttention

RadixCache reuses shared prompt prefixes, reducing repeated prefill work for similar requests.

3. AWQ INT4 quantized serving

Lower-precision weights reduce model memory footprint and test serving behavior under quantization.

Profiling Methodology

Common metrics and comparable request patterns across all frameworks.

Tools

- PyTorch Profiler for HuggingFace baseline generation path
- vLLM and SGLang server logs
- nvidia-smi for GPU memory snapshots
- Weights & Biases for run-level tracking

Metrics captured

- TTFT and end-to-end latency
- Total output-token throughput
- Peak / serving GPU memory
- Short prompts vs shared-prefix prompts
- Concurrency sweep: 1, 2, 4, 8, 16

Results - Headline Numbers

Qwen2.5-1.5B-FP16, concurrency 16, NVIDIA L4.

39.1x

higher throughput

28.34 -> 1108.39 tok/s on shared-prefix prompts

4.4x

lower short-prompt latency

8999 ms -> 2045 ms with vLLM

20.6x

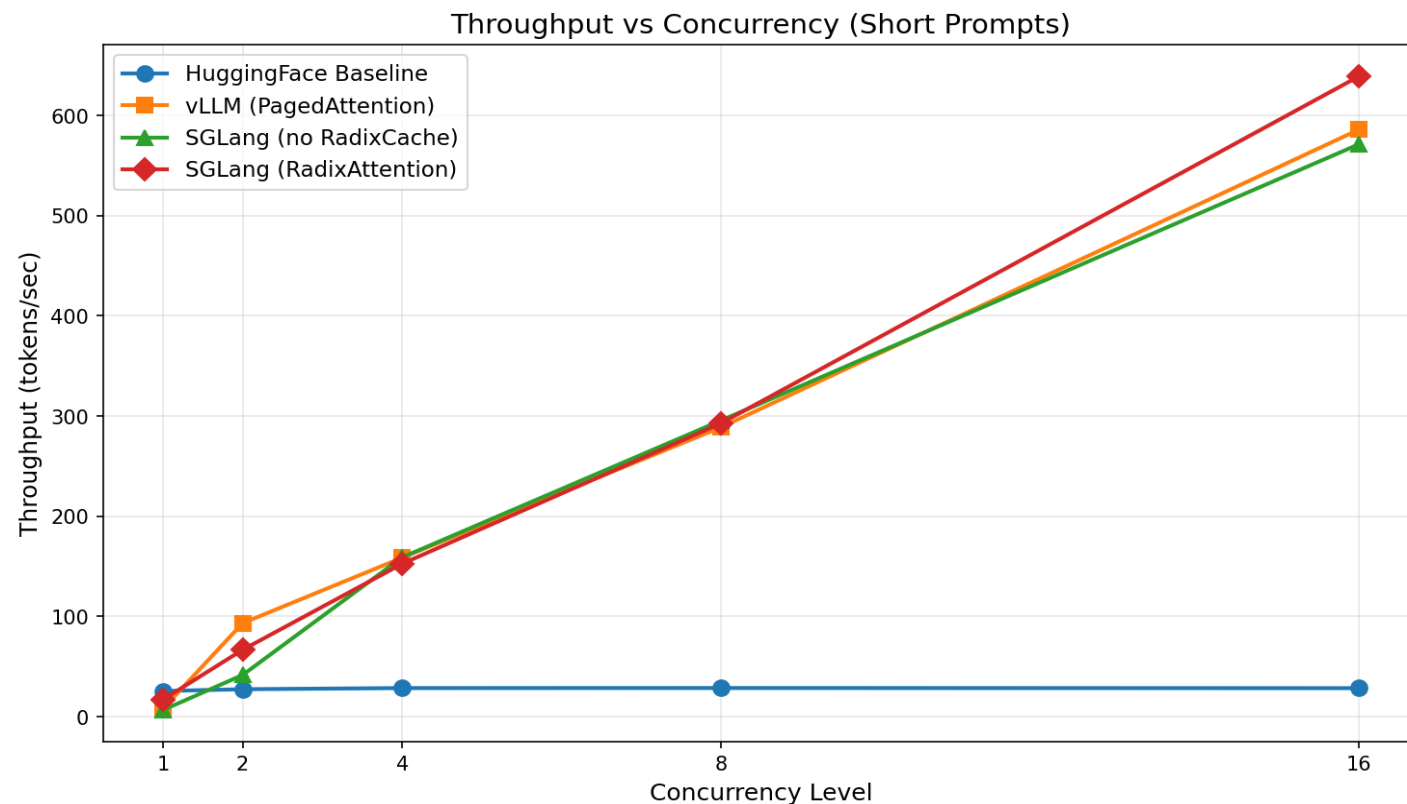
higher short-prompt throughput

28.45 -> 585.90 tok/s with vLLM

Measurements use deterministic generation, max_new_tokens=256, and concurrency levels 1-16.

Results - Before vs. After

Throughput scales dramatically once requests are served by optimized runtimes.



Takeaway

- HuggingFace baseline remains sequential and flat under concurrent load.
- vLLM and SGLang exploit serving-time batching and KV-cache management.
- SGLang is strongest when requests share long prompt prefixes.

Profiler Evidence

PyTorch profiler isolates where baseline generation spends GPU time.

PyTorch Profiler Summary (HF baseline)

Finding	Evidence / Meaning
Matrix multiply dominates	aten::mm $\approx$ 190 ms GPU time
Attention is also costly	FlashAttention + cuBLAS GEMV $\approx$ 195 ms
Baseline repeats work	16 requests run as separate forward passes
Batching reduces idle time	vLLM combines sequences into shared kernel calls
Per-token decode cost drops sharply	HF $\approx$ 35 ms/token $\rightarrow$ vLLM $\approx$ 1.7 $\rightarrow$ SGLang $\approx$ 0.9

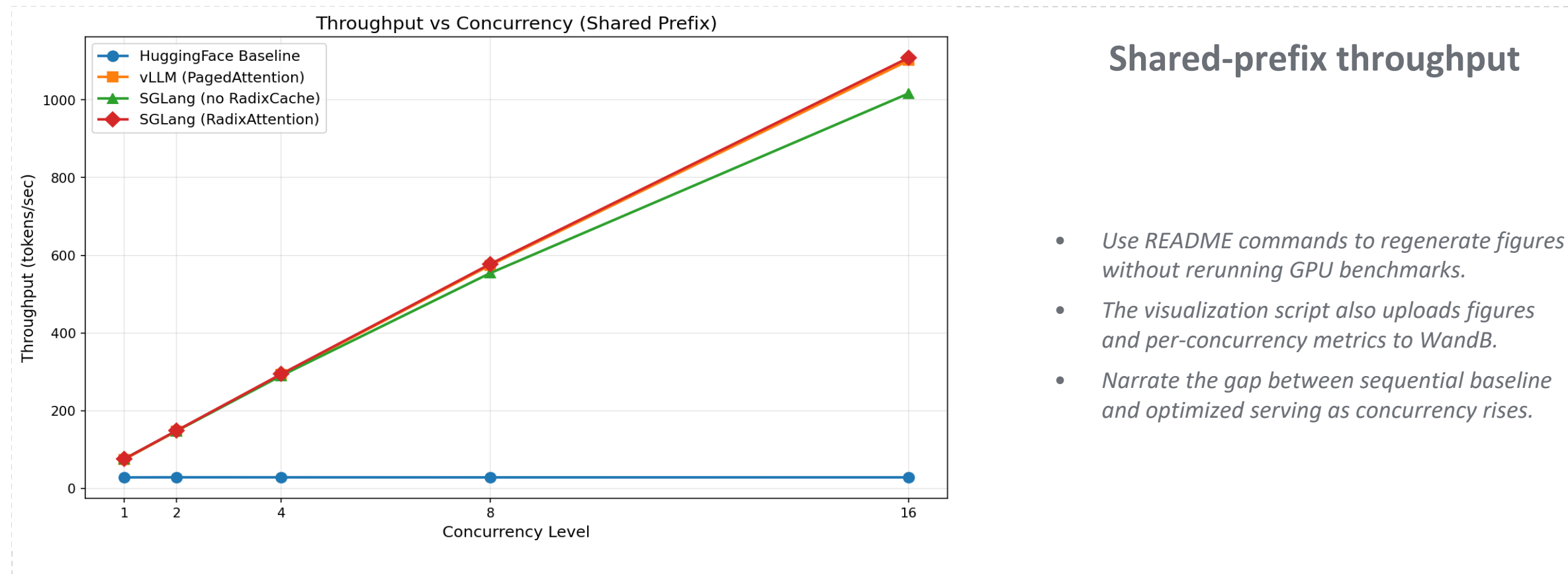
Top kernels: aten::mm, aten::addmm, GEMV, FlashAttention forward, elementwise ops.

What to point at

- Baseline generation pays repeated matrix multiply and token-by-token decode costs.
- Serving frameworks reduce overhead through batching and cache-aware scheduling.
- Shared-prefix prompts reveal the benefit of prefix reuse in SGLang.
- GPU memory increases because serving frameworks reserve KV-cache space.

Demo / Visualization

Static fallback: generated figures from existing benchmark JSON files.



Lessons Learned

The highest gains came from serving-time systems design rather than model changes.

What worked

- Continuous batching made vLLM and SGLang much faster than the sequential baseline.
- Shared-prefix prompts clearly exposed RadixAttention benefits.
- Separate environments avoided dependency conflicts across frameworks.

What didn't work

- One shared Python environment was brittle because serving frameworks pin different Torch stacks.
- Profiling server internals directly was more invasive than profiling the baseline path.
- Raw throughput alone was insufficient; TTFT and memory were needed to explain tradeoffs.

Next Steps

Extend the benchmark to longer contexts and more realistic request traces.

- Measure GPU utilization and queueing behavior over longer steady-state windows.
- Add TensorRT-LLM / TGI comparisons and larger models when more GPU memory is available.
- Evaluate quality and cost tradeoffs for quantized serving beyond AWQ INT4.

Teamwork & Contributions

Team responsibilities and artifacts.

Member	Primary contributions	Tools / artifacts
Yimeng Ye	Benchmarking pipeline, experiment execution	scripts/run_all*.sh, results JSON
Can Yang	Serving framework integration, README and submission organization	src/bench_vllm.py, src/bench_sglang.py, README
Yuxuan Wang	Profiling and visualization	src/bench_profile.py, src/visualize.py
Jiawei Wang	Model/configuration sweep and result validation	configs/config.py, multi-model results
Zhijing Wu	Report, experiment tracking, documentation	WandB dashboard, deliverables/repost.pdf

Thank you

Questions?

GitHub: https://github.com/lennonYe/6998_project

Tracking: <https://wandb.ai/shvpz8ctzd-1/llm-serving-benchmark>

Backup - Experimental Setup

Experiment settings used for the reported L4 rerun.

Setting	Baseline	Optimized
Serving stack	HuggingFace Transformers	vLLM / SGLang
Precision	FP16	FP16 and AWQ INT4
Concurrency	Sequential simulation: 1, 2, 4, 8, 16	Concurrent HTTP requests: 1, 2, 4, 8, 16
Prompt sets	Short Q&A + shared-prefix code analysis	Same prompt sets
Max new tokens	256	256
Hardware	1x NVIDIA L4 24 GB	1x NVIDIA L4 24 GB
Software stack	PyTorch 2.6, Transformers 4.51	vLLM 0.8.4, SGLang 0.4.6